



CMPE491 – High-Level Design Report
HorusEye – AI-Based Exam Proctoring System
2025

Team Members:

Çağla Abazaoğlu – 10051217210

Gizem Nur İpek – 46162949900

Taha Kürşat Öztürk – 15709032030

Ali Sahil – 14596166750

Tuğba Hilal Kırer – 10015328662

Contents

1. Introduction.....	3
1.1 Purpose of the System:.....	3
1.2 System Use Cases.....	3
1.3 Design Goals	5
1.4 Definitions, Acronyms, Abbreviations	5
1.5 Overview.....	6
2. Current Software Architecture.....	6
3. Proposed Software Acrhitecture	7
3.1. Overview	8
3.2 Subsystem Decomposition.....	11
3.3 Hardware/Software Mapping.....	13
3.3.1.1Hardware Components (planned).....	13
3.3.1.2 Software Components	14
3.3.1.3 Mapping	16
3.4 Persistent Data Management.....	16
3.5 Access Control and Security	18
3.6 Global Software Control.....	18
3.7 Boundary Conditions.....	21
4. Subsystem Services.....	22
4.1 Exam Session Management Services.....	22
4.2 Video Ingestion & Stream Services.....	22
4.3 AI Processing Services.....	22
4.4 Incident & Evidence Services.....	22
4.5 Alert & Notification Services	22
4.6 Reporting & Analytics Services.....	22
4.7 User & Role Management Services.....	22
5. Glossary.....	23
6. References.....	23

1. Introduction

1.1 Purpose of the System:

HorusEye is an AI-based remote exam proctoring tool intended to promote integrity in remote or offline exams. Conventional human remote invigilation methods and simple webcam recording systems are limited by issues of scalability, objectivity, evidence documentation, and timeliness. HorusEye will attempt to overcome all these issues by remotely monitoring examination rooms through webcam channels, identifying irregular acts (such as irregular eye movements or device operating, whispering, and communication acts), and alerting supervisors through an encrypted results panel. Such irregular acts are intended to be recorded along with time stamps and pictorial evidence.

1.2 System Use Cases

Figure 1.1 presents the primary use cases and actors of the HorusEye system. The system supports three main actor types: Administrators who manage system configuration and exam sessions, Supervisors who monitor exams and review incidents, and the AI System which operates autonomously to detect and flag suspicious activities. The use cases are organized into seven functional groups: authentication, session management, AI processing, incident management, alerts, reporting, and monitoring.

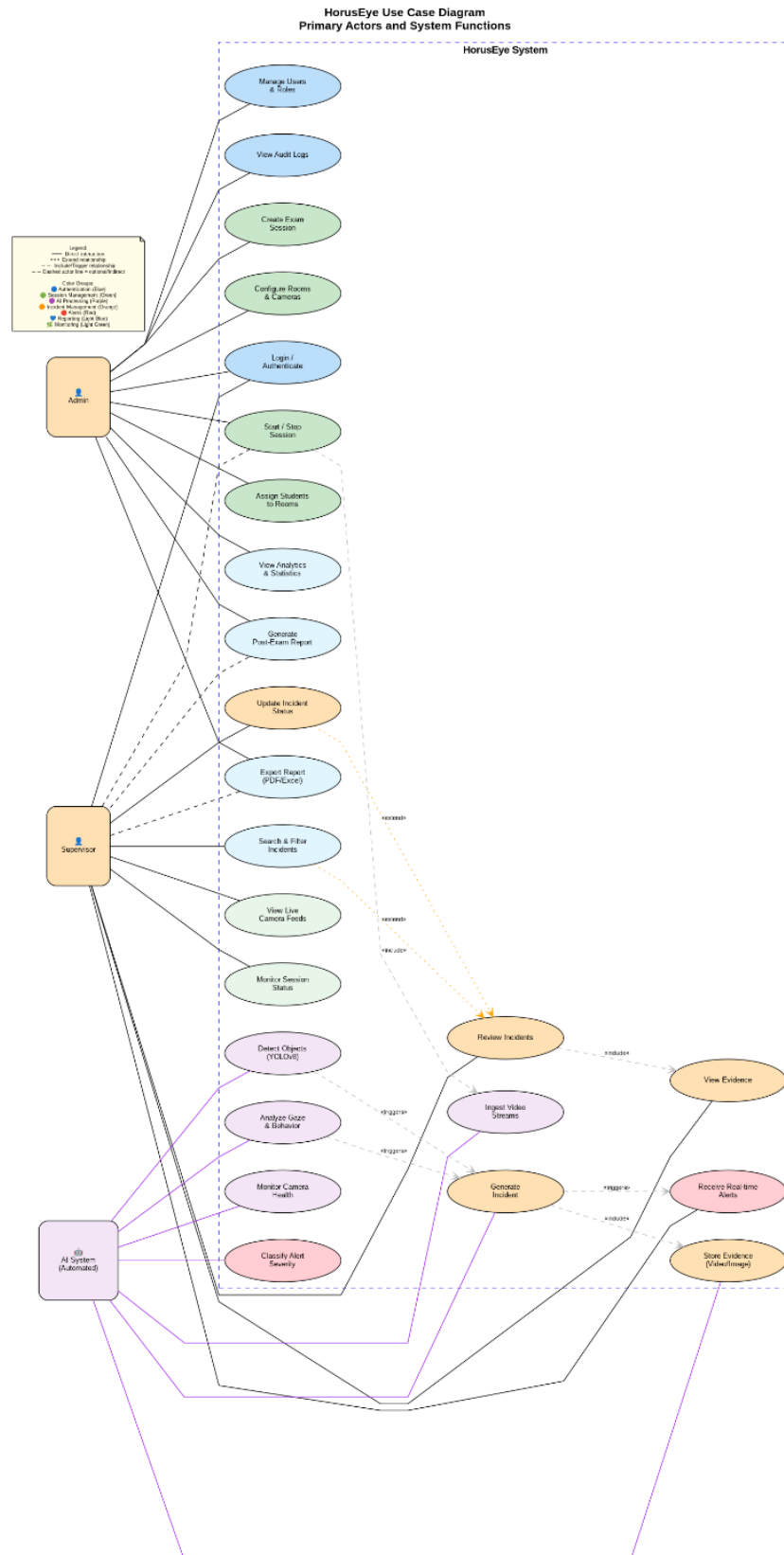


Figure 1.1

Figure 1.1: HorusEye Use Case Diagram showing three primary actors and 24 use cases grouped by functional area. Include and extend relationships illustrate dependencies between use cases.

1.3 Design Goals

- **Completeness & Accuracy:** It is designed to achieve at least 85% of its detection accuracy under normal conditions, with mechanisms that reduce false positives/negatives through confidence scores and configurable thresholds.
- **Real-Time Responsiveness:** HorusEye is designed to generate low-latency alerts during live exams using real-time communication-for example, WebSocket-so supervisors can react with speed.
- **Scalability:** This architecture allows for multiple exam rooms, multiple cameras, and a number of monitoring sessions without having any significant performance issues.
- **Compliance with Safety & Privacy Legislation (KVKK/GDPR):** The system shall allow for the security of data in transit and storage, access control through the use of roles based on RBAC, handling of personal data in accordance with responsible privacy requirements.
- **Auditability & Evidence-Based Review:** HorusEye is designed to store timestamped incident records and associated evidence so that decisions can be reviewed transparently after the exam.
- **Usability for Non-Technical Staff:** The design of the dashboard allows supervisors, who are not technical people, to easily monitor feeds, triage alerts by severity of incident, and review evidence on their own.

1.4 Definitions, Acronyms, Abbreviations

- **Artificial Intelligence (AI):** Techniques that allow automatic decision-making based on data.
- **Computer Vision:** AI area dealing with the analysis of images/videos for object/behavior detection.
- **YOLOv8:** This is another object detection model that is planned to support the detection of devices and all relevant objects.
- **OpenCV:** A library designed for video processing and computer vision-related tasks.
- **TensorFlow:** Framework planned for deep learning / behavior modeling modules.

- REST API: HTTP-based interface to non-real-time operations; it allows access to reports, history, and configuration.
- Websocket: Channel intended for real-time alerts, state updates, and live events related to monitoring.
- RBAC: Role-Based Access Control-Admin/Supervisor permissions.
- KVKK/GDPR: Data protection regulations that are relevant to the storage and processing of personal data.

1.5 Overview

HorusEye is intended to be an AI-driven software-intensive monitoring system. Ingestion and processing of the camera feed will be done through an AI module. The alarms will be converted to incidents along with confidence/risk scores and sent to the authorities over the web interface." It appears that this system is intended for both online analysis during "live exams" as well as offline analysis after the exam, using recorded videos. Also, the system uses "microservices for modularization." This suggests that the system breaks down tasks into multiple services, thereby overcoming "concerns of streaming, inference, notification, and storage.

2. Current Software Architecture

The current state of examination proctoring is usually founded upon:

- Personal invigilation, since personal invigilation lacks the ability to observe students closely to detect suspicious behavior.
- Non-intelligent online monitoring systems, which primarily utilize webcams for recording videos without analyzing gaze or object usage or sending alerts based on artificial intelligence.
- Weak evidence support, which does not record incidents with time-tamps and search metadata.
- Scalability in larger examinations and cost considerations for multi-room circumstances, because of manual human monitoring.
- Privacy and compliance issues related to unclear handling and retention practices.

HorusEye is intended to fill these gaps by leveraging automated detection, systematic logging of evidence, and scalable monitoring features.

3. Proposed Software Architecture

Figure 3.1 illustrates the data flow within the HorusEye system at the context level (DFD Level 0). The diagram shows eight core processes, five data stores, and three external entities. High-volume real-time data flows (shown with bold lines) are distinguished from standard data flows and cached data (shown with dashed lines). This overview establishes the foundation for the detailed architectural design presented in the following sections.

- Deployment & Operations Layer: Containerized deployment with proper subsystem segregation for maintainability and scalability.

Design principles and validity (why this works):

- Separation of Concerns: It has a single responsibility, and this makes it easier to test and modify.
- Scalability through decomposition: AI inference server can be scaled independently from UI/reporting server.
- Real-time responsiveness: Real-time push capabilities from the WebSocket protocol increase responsiveness over polling.
- Auditability: Evidence-logging/reporting has been considered a first-class functionality, instead of an after
- Security-by-design: Role-based access control(RBAC), encrypted transport, and regulated data handling are built into the architecture.

Figure 3.2 provides a comprehensive view of the HorusEye system architecture, depicting all major layers and their interactions. The architecture follows a layered approach with clear separation of concerns: the Presentation Layer handles user interfaces, the Backend Services Layer manages business logic and orchestration, the AI Processing Layer performs real-time video analysis, and the Data Layer provides persistent and cache storage. Communication protocols between components (REST, WebSocket, RTSP, WebRTC) are explicitly shown to clarify integration points.

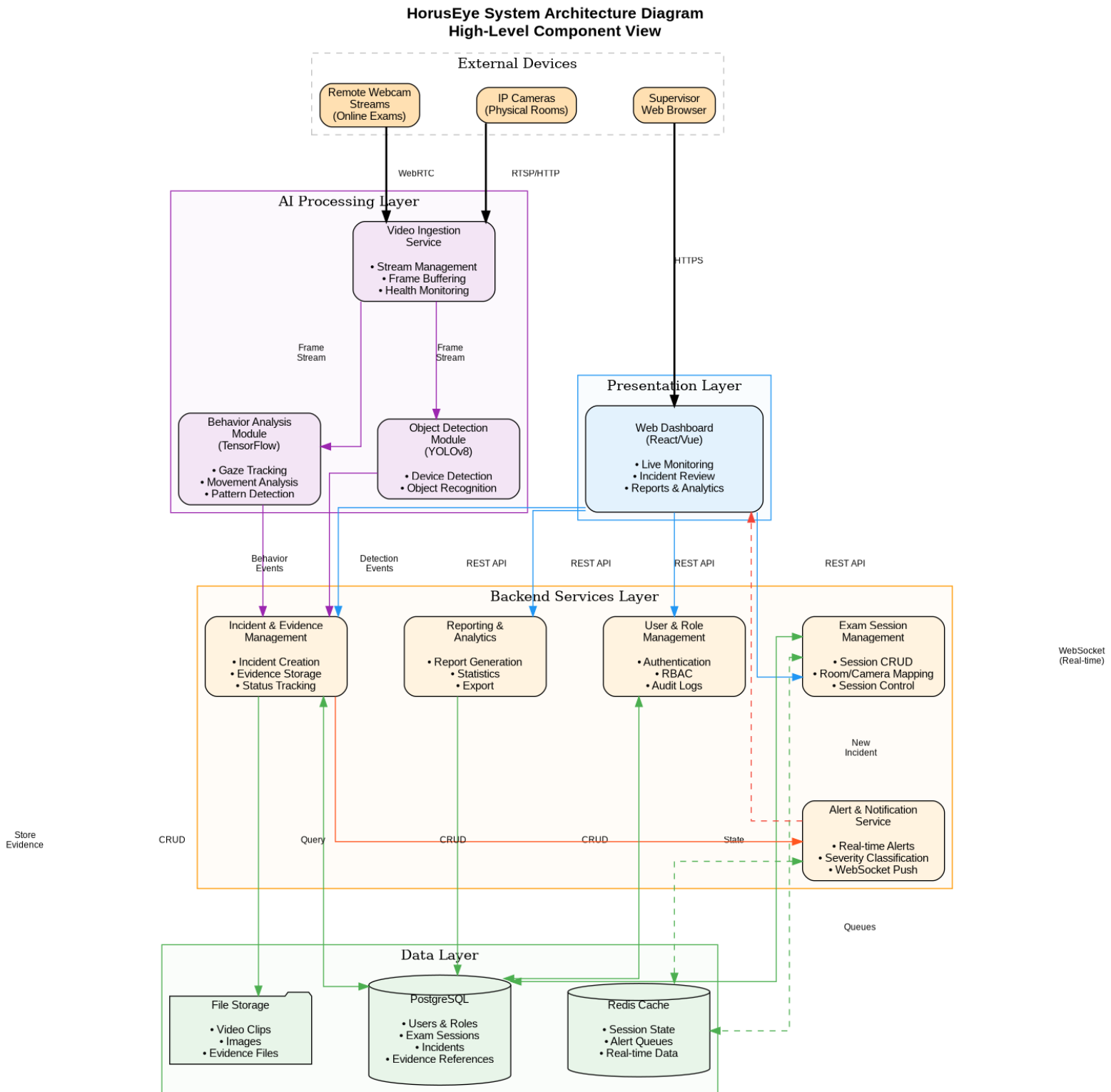


Figure 3.2

Figure 3.2: HorusEye System Architecture showing the layered design with presentation, backend services, AI processing, and data layers. Communication protocols between components are indicated with labeled arrows.

Creativity/novelty (what is new vs. typical solutions):

- The integration of real-time AI detection, evidence organization, and risk analysis into a single process for both live response and post-exam analysis, as opposed to simply logging.
- It supports both real-time and offline analysis modes through one architecture design.

3.2 Subsystem Decomposition

HorusEye is broken down into the following subsystems:

- Exam Session Management Subsystem
- Generates/exits exam sessions, relates rooms/cameras/students, handles life
- Executes session rules and acquires monitoring metadata.
- Video Ingestion & Stream Management System
- Links to camera feeds (physical rooms) or to remote streams (online rooms).
- Buffers frames of a feed, manages health of said feed/stream, and relays frames to AI processing.
- AI Processing Subsystem
- Runs planned CV/DL models (e.g., YOLOv8 + gaze/behavior analysis
- Fires detection events with confidence values and detected categories (phone, gaze diversion, etc.).
- Incident & Evidence Subsystem
- Translates detection events into Incidents with standard fields (type, date, student, confidence, risk).
- Produces or cites evidence (clip/image) for traceability purposes during audits.
- Alert and Notification System
- Sends live alerts to the dashboard via WebSocket.

- Allows severity classification (e.g., Critical, High, Warn) and helps with
- Reporting & Analytics Subsystem
- It generates post-examination reports, which summarize the incidents, the rate of occurrence, categories, levels, and
- Provides support for filtering by session, student, or time range for review workflows.
- User & Role Management Subsystem
- Manages the accounts of the supervisors/administrators, as well as the authentication
- RBAC for sensitive operations (configuration changes, evidence access).
- Data Persistence & Cache Subsystem
- Stores data in PostgreSQL structurally.
- Utilizes caching/queue systems (for example, Redis) for low-latency alert/state

Figure 3.3 presents the internal structure of each subsystem, breaking down the components into three architectural layers: Controllers (which handle external requests), Services (which implement business logic), and Models (which represent data entities). Dashed lines indicate internal dependencies within each subsystem, while solid lines show interfaces and data flows between subsystems. This component-level view demonstrates how the modular architecture supports independent development, testing, and scaling of subsystems.

HorusEye Component Diagram
Subsystem Internal Structure

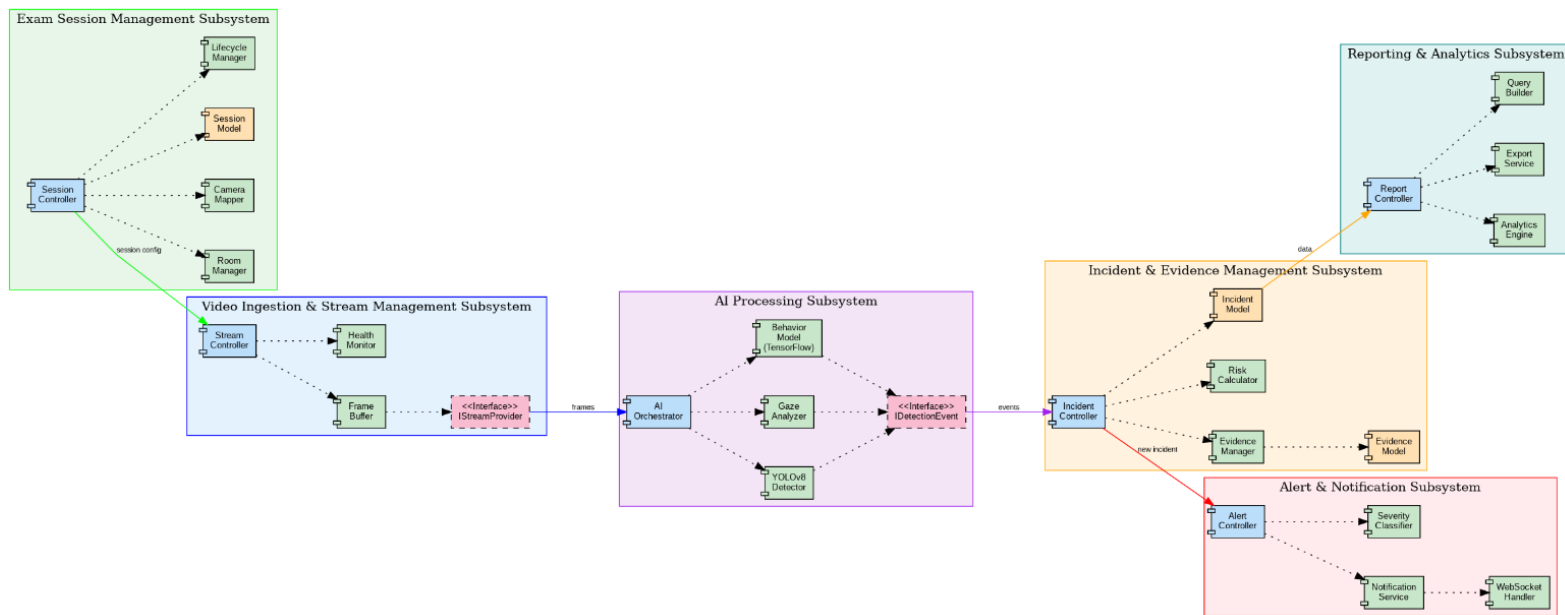


Figure 3.3

Figure 3.3: Component Diagram illustrating the internal structure of each subsystem including controllers, services, and models. Dashed lines represent internal dependencies while solid lines show external interfaces.

3.3 Hardware/Software Mapping

3.3.1.1 Hardware Components (planned)

- Cameras / Video Sources: Exam Rooms/IP Cameras or Online/Webcam sources for online sessions.
- Processing Node (optional GPU): The machine on which are executed the services of inference of Artificial Intelligence.
- Backend Server: Handles REST/WebSocket services as well as overall application logic.
- Database/Cache Server: It holds the PostgreSQL and Redis databases (can run both together on the same machine for small implementations).

- Supervisor Client Devices: Web Browsers Used by Supervisors/Admins.

3.3.1.2 Software Components

- Front-end: Dashboard based on React/Vue (final choice TBD).
- Backend: Python Microservices using REST + WebSockets.
- Components for Artificial Intelligence Layer: OpenCV + YOLOv8 + TensorFlow for detection and modeling.
- Data Layer: PostgreSQL for persistent records and Redis for real-time state and queues.
- Deployment: Using Docker containers for consistency in environment and scaling.

Figure 3.4 depicts the physical deployment architecture of the HorusEye system across four dedicated servers. The containerized approach using Docker ensures environment consistency and simplifies scaling. Each server is allocated based on resource requirements: the Application Server hosts frontend and backend services, the AI Processing Server (GPU-enabled) handles computationally intensive detection tasks, the Database Server manages persistent and cached data, and the File Storage Server stores video evidence. Network connections and port mappings are specified to clarify the deployment topology.

**HorusEye Deployment Diagram
Container & Infrastructure View**

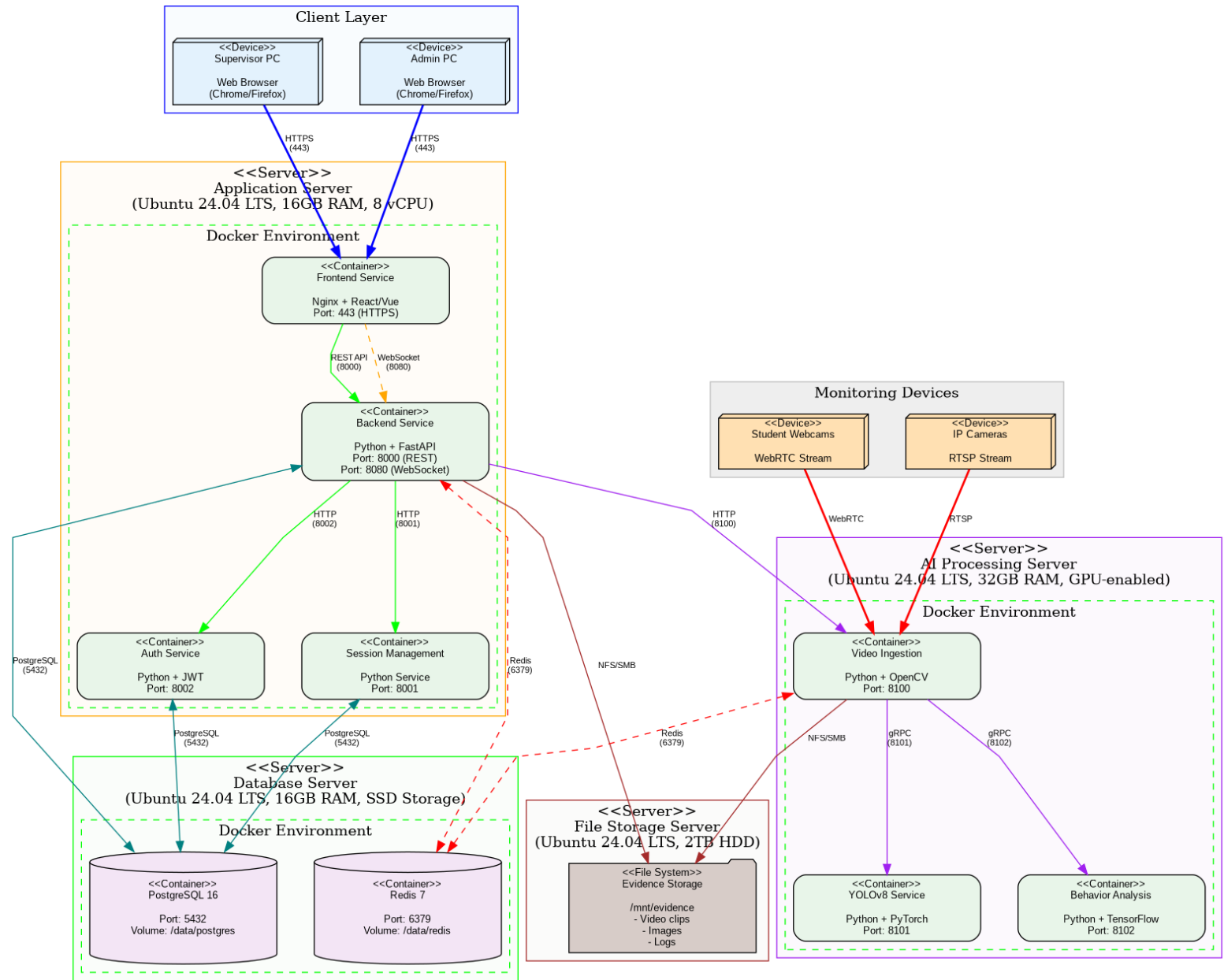


Figure 3.4

Figure 3.4: Deployment Diagram depicting the Docker-based containerized architecture across multiple servers. Port numbers and network protocols are specified for each connection.

3.3.1.3 Mapping

- Cameras transmit the data to the Video Ingestion Service (network).
- Video ingestion distributes the frames to the AI Processing Service (either local network or internal service mesh).
- AI outputs detection events to Backend/Incident Service.
- Backend stores incidents in PostgreSQL databases, maintains real-time states in Redis, and sends alerts to the Frontend through WebSockets.
- The supervisors will access the system through the HTTPS interface and get real-time notifications without the need for polling.

3.4 Persistent Data Management

HorusEye is intended to handle:

- **Users/Roles/Access logs**
- **Exam sessions** (room, time, camera mapping, status)
- **Incidents** (event type, timestamp, confidence, risk, student/session association)
- **Evidence references** (paths/IDs to video clips or images)

Figure 3.5 presents the complete database schema with nine interconnected tables. Primary keys are marked with  symbols, foreign keys with  symbols, and cardinality is shown on relationship lines. The schema is designed to support complete incident tracking with full audit trail capabilities. Key relationships include one-to-many associations between sessions and incidents, incidents and evidence, and users and multiple entity types for comprehensive access logging.

HorusEye Database ER Diagram PostgreSQL Schema

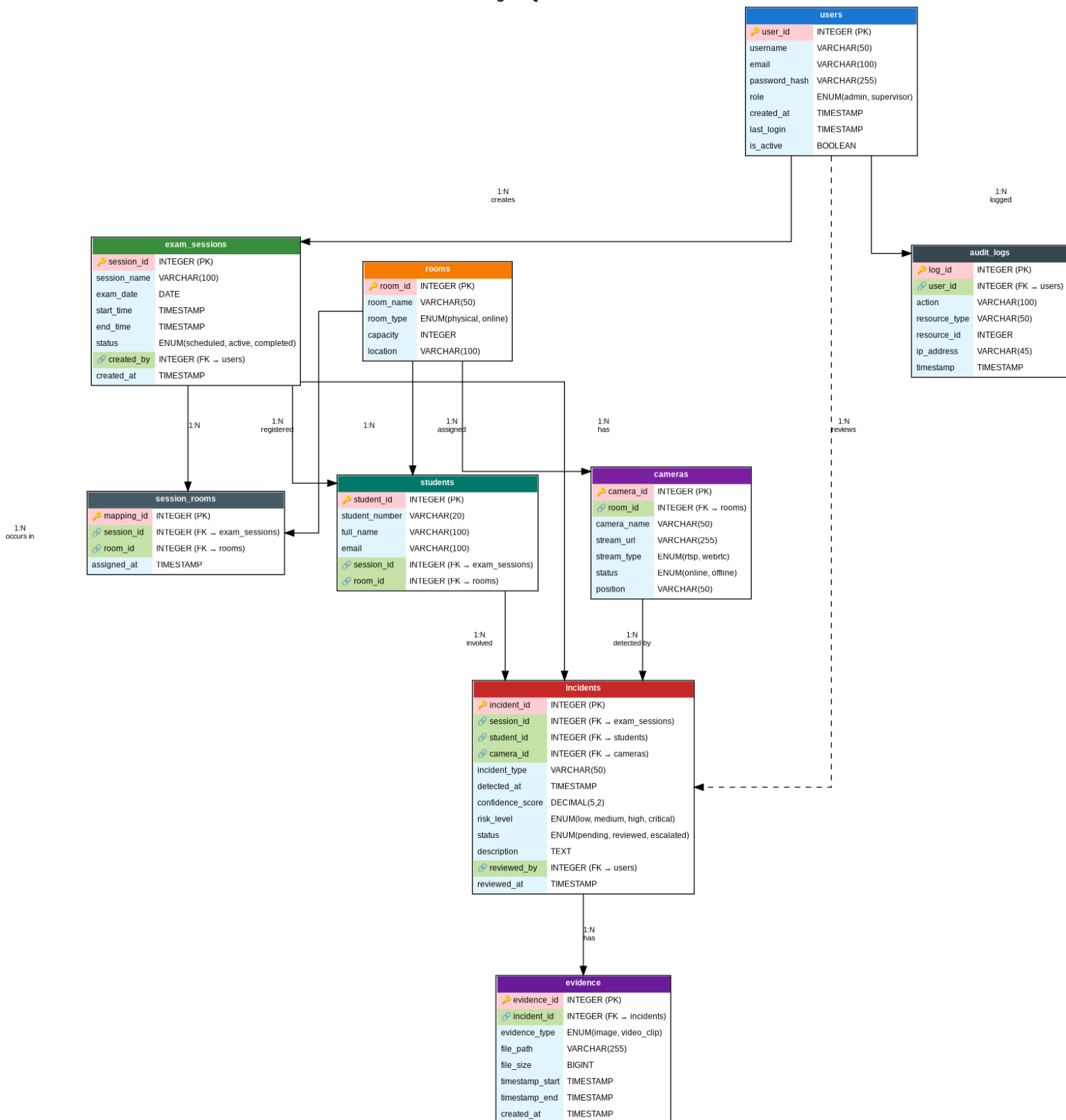


Figure 3.5

Figure 3.5: Database Entity-Relationship Diagram showing nine main tables with primary keys, foreign keys, and their relationships. The schema supports complete incident tracking and audit trail.

PostgreSQL is chosen for structured storage and querying for reporting. Evidence data is intended to be stored as files/objects with database references, not necessarily as relational data. The plan is to use Redis for temporary storage: real-time session status, warning queues, and low-latency synchronization.

Design principle: Persistent truths are maintained in PostgreSQL. Except where latency is a major concern, volatile truths are maintained in Redis.

3.5 Access Control and Security

- Authentication & Authorization (planned):
 - User login process for supervisors/admins.
 - RBAC: The Admin is responsible for configuration/user management, while the Supervisor is in charge of session
- Transport & Storage Security:
 - HTTPS/TLS for communication.
 - Encryption of sensitive storage data, secure management of credentials, and least privilege access.
- Privacy & Compliance (KVKK/GDPR):
 - Data retention policies (delete after configurable period).
 - Access logging and audit trails for evidence access.
 - Minimize data revelation (authorized personnel can view evidence data.)
- Bias & Fairness Considerations:
 - To decrease unjust flagging, using confidence scoring, threshold tuning, and supervisor review (human-in-the-loop).

3.6 Global Software Control

The global control relates to the overall control of all subsystems.

Exam lifecycle control (planned):

1. Initialization: Admin configures rooms/cameras; Admin logs in.
2. Session Preparation: Choose the desired exam session → Session setup for the exam → Validate the connection of the cameras.
3. Monitoring Start: Streams begin; AI inference starts; events are produced.
4. Event Handling: Detection event → Incident creation → Risk update → Alert push → Evidence reference creation.

Figure 3.6 illustrates the detailed sequence of operations during incident detection and alert generation, which represents the core functionality of the system. The diagram traces the complete flow from camera feed ingestion through AI detection, incident creation, evidence extraction and storage, to final alert delivery via WebSocket. The 16-step sequence demonstrates the system's real-time processing capabilities with typical latency under 2 seconds from detection to dashboard alert.

Incident Detection & Alert Flow Sequence Step-by-Step Process from Detection to Dashboard Alert

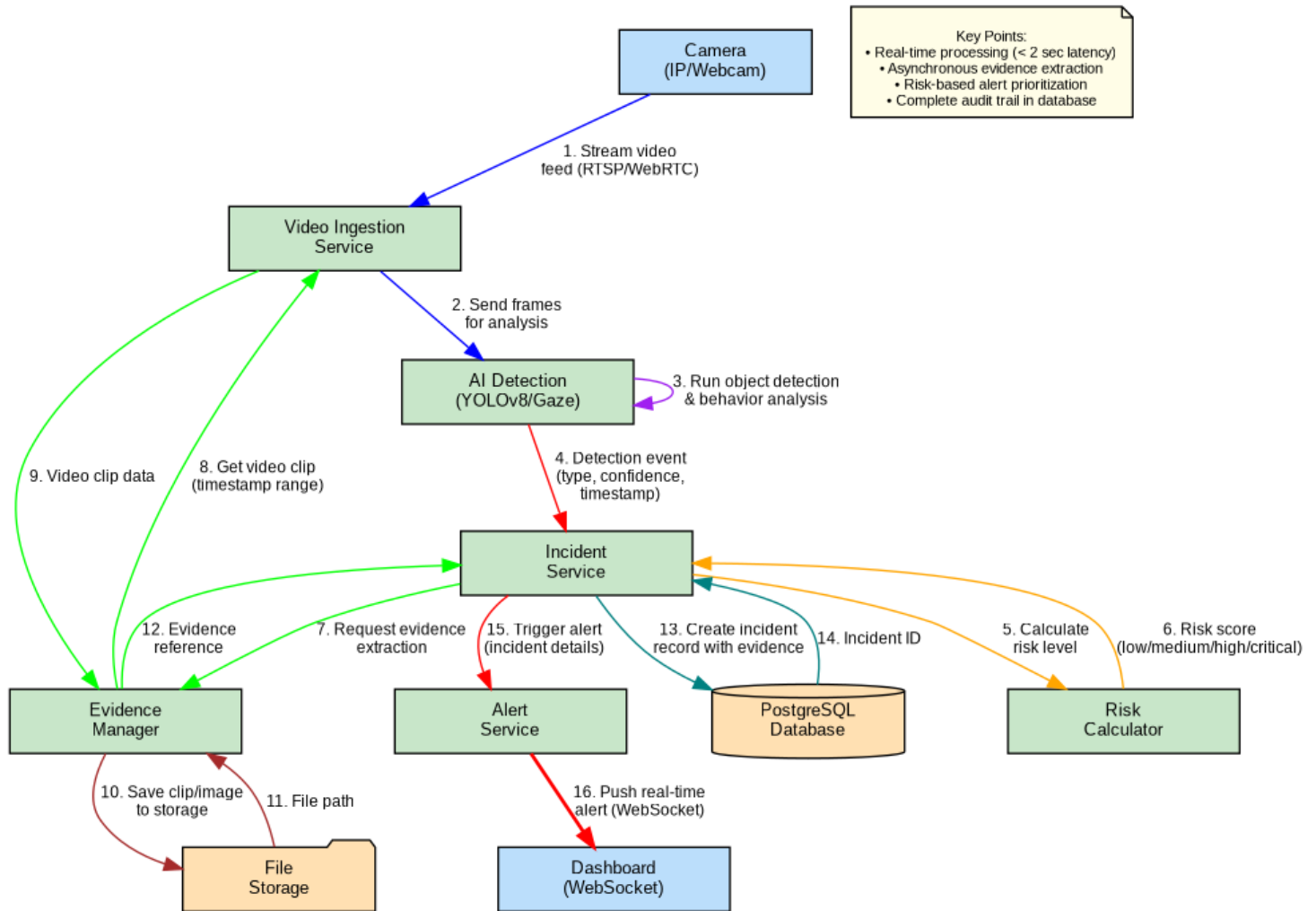


Figure 3.6

Figure 3.6: Sequence Diagram of the Incident Detection and Alert Flow showing 16 sequential steps from camera feed to real-time dashboard alert, including evidence extraction and storage.

5. Session End: Monitoring stops, reporting service collates data and provides post-exam report.
6. Review: The supervisor examines incidents/evidence. They also label the reviewed/escalated incidents. Finally exports report if needed.

Validity of control strategy:

This is because event-driven applications remain responsive.

It has a session-based lifecycle, making behavior analysis and reporting easier and consistent across the board.

3.7 Boundary Conditions

- Initialization conditions:

When a required camera stream is not available, it is not possible to begin the session or begin with partial coverage with warnings.

- Normal operation conditions:

If the confidence level in the results of the AI system is below the threshold level: the event will be low severity or queued for aggregation.

- Failure handling:

Camera/stream failure: The stream health flag goes on along with a warning on the dashboard and continue to monitor remaining streams.

Network problems: Temporary buffering/queueing; resume when the connection is restored at best effort.

Failure in an AI service: Degrade gracefully, which for us means: stop the inference but keep recording and maintain state; log errors for later analysis.

DB failure: Queue incidents temporarily by utilizing cache/queue, retry persistence. Avoid data loss when possible.

- Termination conditions:

Session end initiates graceful shutdown of streaming/inference, completes incident writes, and report generation.

4. Subsystem Services

High level service list (implementation independent):

4.1 Exam Session Management Services

- Create/Update/Delete exam sessions (admin)
- Assign rooms/cameras/students
- Start/Pause/End session
- Retrieve session status (for dashboard)

4.2 Video Ingestion & Stream Services

- Register camera stream
- Fetch live frames / stream metadata
- Monitor stream health (online/offline, FPS, latency)

4.3 AI Processing Services

- Run object detection (device/phone/earbuds where applicable)
- Run gaze/behavior analysis
- Produce detection events with confidence values

4.4 Incident & Evidence Services

- Create incident from detection event
- Associate incident with student/session/camera
- Store evidence reference (clip/image)
- Update incident status (reviewed/escalated)

4.5 Alert & Notification Services

- Push alert to dashboard (real-time)
- Categorize severity and maintain alert queue
- Acknowledge/resolve alert actions

4.6 Reporting & Analytics Services

- Generate post-exam report
- Summarize incidents (counts, categories, risk)
- Filter/search incidents for review
- Export report (format TBD)

4.7 User & Role Management Services

- Authenticate user

- Manage roles/permissions (admin)
- Audit access logs (evidence access)

5. Glossary

Incident: This is a recorded suspicious event that contains information regarding the time, type, and evidence related to the

Evidence: Video clip/image relating to an incident to be verified.

Confidence Score: A measure of the certainty of the AI regarding the results of a detection task.

Risk Level: The aggregated severity of incidents based on patterns of incidence.

Dashboard: Provides web user interface for the purpose of monitoring, notifications, analysis of evidence, as well as reporting

6. References

Bernd Bruegge, Allen H. Dutoit, *Object-Oriented Software Engineering: Using UML, Patterns, and Java*, 2nd Ed., Prentice-Hall, 2004.

HorusEye Analysis Report (CMPE491 – Analysis Report, 2025).

HorusEye Project Specifications Report (CMPE491 – Senior Project I, 7 Nov 2025).